Queries, scores and filters

What happens to a customer's words before, during and after the search

These notes go deeper into three things that `learn.pdf` only touches: why a question is embedded differently from a product, what the score on the page means, and where filters such as "under 5000 kr" come from. Every number in them was measured on this store's 500 products.

contents

1 Two kinds of text **2**
1.1 A librarian . . . 2
1.2 What goes wrong without it . . . 2
1.3 Measured on all the test questions . . . 3

2 Other task types **4**

3 What the score means **5**
3.1 Today: the keyword placeholder . . . 5
3.2 After your search works: closeness in meaning . . . 6

4 Where filters come from **6**
4.1 From the page . . . 7
4.2 From the words . . . 7
4.3 Two kinds of model . . . 7
4.4 Structured output . . . 7
4.5 It needs testing too . . . 8

A All 20 graded questions **10**

1 Two kinds of text

A search always compares two different kinds of text. On one side is the customer's *question*, which describes a need:

> *something soft to sleep on if I sleep on my side*

On the other side is the *product text*, which describes a thing:

> *Memory foam mattress with soft firmness, 140x200 cm. Soft and cosy.*

The two share almost no words. The customer does not know the product is called "memory foam, soft firmness"; they only know their problem. Yet the second text is a good answer to the first.

1.1 A librarian

Think of a library. The books on the shelf are described by what they *are*: "A guide to growing tomatoes in small gardens." The visitors ask about what they *need*: "how do I get vegetables on my balcony?" A good librarian does not look for the book whose description sounds most like the visitor's sentence. They ask themselves which book *answers* it.

The embedding model can work in either way. You tell it which with one setting, the *task type*:

task type: a label sent with the text that says what role the text plays in the search.

- `RETRIEVAL_DOCUMENT`: "this is a book on the shelf". The vector describes what the thing is. The loader used this for the 500 products.
- `RETRIEVAL_QUERY`: "this is a visitor asking". The vector points toward what would answer them. Your `embed_query` uses this for the question.

The model has been trained on many pairs of a question and its right answer, so that a question embedded as a query lands close to the documents that answer it, not close to texts that merely look like it.

1.2 What goes wrong without it

To see the difference, the same question was embedded both ways and compared with the same 500 product vectors. Table 1 shows the five best products each time.

as a query (right)			as a document (wrong)		
✓	0.712	LYNGESKOV mattress	×	0.752	RAVAHOLM pillow
✓	0.704	VORSBY mattress	×	0.751	ASKESTED pillow
✓	0.704	BØGEBY mattress	✓	0.751	LYNGESKOV mattress
✓	0.702	ASKEAGER mattress	✓	0.748	VORSBY mattress
✓	0.702	RINGSGAARD mattress	×	0.748	SOLGAARD pillow
precision@5 = 1.0			precision@5 = 0.4		

Table 1 The five best products for "something soft to sleep on if I sleep on my side", with the question embedded each way. All the mattresses listed are soft.

As a document, the question finds two pillows whose whole description is their name: "RAVAHOLM pillow." and "ASKESTED pillow.". They are short and about sleep, like the question, so they *look* alike. They do not answer it. As a query, the question finds five soft mattresses, which is what a side sleeper needs.

Figure 1 draws the same idea. The real vectors have 768 numbers each; the sketch flattens them to two, so only the positions relative to each other mean anything.



Figure 1 A sketch, not a measurement. The same sentence lands in a different place depending on its task type. As a document it sits near texts that look like it; as a query it sits near texts that answer it.

1.3 *Measured on all the test questions*

The same comparison was run on the 20 graded test questions, the ones `check.py` reads from the file `data/eval/product_queries.json`. The full list is in Appendix A; Table 2 has the averages.

	precision@5			
Question embedded as	meaning (15)	language (5)	gap	top score
`RETRIEVAL_QUERY`	**0.99**	**0.96**	**+0.043**	0.72
`RETRIEVAL_DOCUMENT`	0.93	0.80	+0.021	0.80

Table 2 Averages over the 20 graded questions, one run each. Both ways find at least one right product in the top five for every question (hit@5 is 1.00 for both). The *gap* is the score of the best right product minus the score of the best wrong one.

Three things stand out.

1. **The query type is better**, and most clearly on the Danish and German questions: 0.96 against 0.80. “Matratze gegen Rückenschmerzen” (German for “mattress against back pain”) scores 1.0 as a query and 0.6 as a document.
2. **The right answers stand out twice as clearly.** The *gap* doubles, from 0.021 to 0.043. A bigger gap means a small change in wording is less likely to push a wrong product to the top.
3. **Higher scores, worse results.** As a document, the top scores are higher (0.80 against 0.72) while the results are worse. Section 3 explains why this is not a contradiction.

gap: how far the best right product scores above the best wrong one. Bigger is safer.

Products are *documents*; the customer’s words are a *query*. Embed each with its own task type, and the question is matched with what answers it, not with what looks like it.

2 Other task types

The store is *asymmetric*: a short need on one side, a product description on the other. Not every application is like that.

Take a project management tool where every past project has a set of cards: scope, stakeholders, schedule, budget, risks, communication plan. A new project comes in with its own description, and the question is “which past projects were like this one?”. Now both sides are the same kind of text: a project description against project descriptions. The comparison is *symmetric*, and nobody is asking a question.

For that case the model has a separate task type, used on both sides. Table 3 lists the ones the Gemini embedding model offers.

asymmetric: the two sides of the comparison are different kinds of text: a short need on one side, a description on the other.

symmetric: both sides are the same kind of text, and either could be swapped for the other.

Task type	Use it when
`RETRIEVAL_QUERY` + `_DOCUMENT`	a question looks for the text that answers it (this store)
`SEMANTIC_SIMILARITY`	both sides are the same kind of text (similar past projects)
`CLASSIFICATION`	texts are sorted into fixed labels (“high risk”, “low risk”)
`CLUSTERING`	texts are grouped with no labels given (families of projects)
`QUESTION_ANSWERING`	a question is matched with passages that hold the answer
`FACT_VERIFICATION`	a claim is matched with passages that support or refute it

Table 3 Task types of `gemini-embedding-001`. The choice is a starting point; a small test set decides.

One more lesson from the project example: embed each card on its own and compare like with like, the new risk card with past risk cards and the new scope with past scopes. One long text per project blurs everything together, so a project with a similar budget but a different scope would look half-similar to everything.

3 What the score means

Each product card on the page shows a small number, for example *score 0.72*. What it means depends on which search produced it.

3.1 Today: the keyword placeholder

Until `search()` in `api/search.py` works, the API falls back to a simple keyword search (`api/keyword_search.py`) and the page says “keyword placeholder”. Its score is the share of the question’s words that appear in the product’s text:

$$\text{score} = \frac{\text{question words found in the product}}{\text{all question words}}$$

example

The customer types *grey wool rug*: three words.

A grey wool rug	grey, wool, rug: 3 of 3	1.00
A blue wool rug	wool, rug: 2 of 3	0.67
A grey sofa	grey: 1 of 3	0.33
A grey couch	grey: 1 of 3	0.33

It counts words and knows nothing about meaning: a search for *couch* never finds a product called *sofa*.

3.2 *After your search works: closeness in meaning*

Once `search()` works, the score is the *cosine similarity* between the question's vector and the product's vector: the number your matrix product `index.vectors @ question` gives for that product. Higher means closer in meaning. Three things about it are easy to get wrong.

cosine similarity: how closely two vectors point the same way. With vectors of length 1 it is just their dot product.

It is not a percentage. A score of 0.72 does not mean "72 % relevant". On this store, good matches land around 0.70 to 0.80 and poor ones a little lower. The range is narrow because all 500 texts are about furniture and already have much in common.

It only ranks. A score is useful for one thing: deciding, for *one* question, whether product A is closer than product B. A 0.72 for one question and a 0.72 for another cannot be compared.

No fixed cut-off. Because of this, a rule such as "only show products above 0.75" does not work. Table 2 showed why: embedding the question as a document raised every score to about 0.80 while making the results worse. The whole scale moved; the order is what got worse. Figure 2 shows this for the side sleeper question.

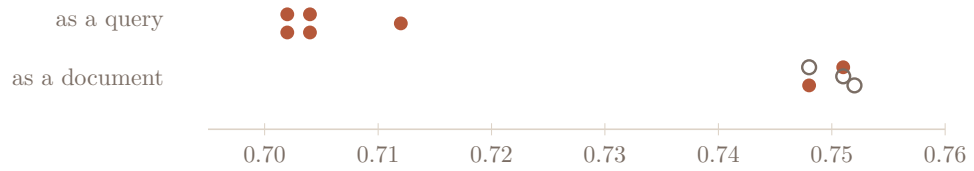


Figure 2 Scores of the top five products for the side sleeper question. Filled dots are right answers, rings are wrong ones. The document row scores higher and still lets three pillows in.

> The score is a ranking signal for one question, not a measure of quality. Compare scores within a question, never across questions or settings.

4 Where filters come from

A *filter* is a rule the results must obey, such as "only sofas" or "at most 5000 kr". Your filter function in `api/search.py` receives them as a `SearchFilters` with two fields, `category` and `max_price`, either of which can be `None` for "not set". It does not care where they came from. There are two places they can come from.

filter: a hard rule that removes products from the results, no matter how close in meaning they are.

4.1 *From the page*

The customer clicks a category chip such as *Sofa*, or picks a maximum price. The page sends those choices to the API next to the question, and the API turns them into `SearchFilters(category="sofa", max_price=5000)`. This is what Step 1 does, and it is exact: a click means what it says.

4.2 *From the words*

A customer may instead type it all in one sentence:

> *A red comfortable sofa under 5000 dkk*

An embedding handles *red*, *comfortable* and *sofa* well, because those are meaning. It cannot handle *under 5000 dkk*. To an embedding, that is a few words with a vague money feel; it has no way to check that 4800 is below 5000, so a sofa at 9000 kr can still come first. This is what the three `needs_filter` test questions show ("sofa under 4000 kr", "sofa that is not leather", "mattress 180x200"), and why `check.py` does not grade them.

The fix is to read the sentence *before* searching, and split it into the part that is meaning and the part that is a rule. That job needs a different kind of model.

4.3 *Two kinds of model*

	Embedding model	Text model
Name	`gemini-embedding-001`	`gemini-3-flash-preview`
Call	`models.embed_content`	`models.generate_content`
Gives back	768 numbers	text, or JSON in a shape you define
Reads "5000" as a number	no	yes

Table 4 The embedding model turns text into a vector; only the text model can read the sentence and fill in a form.

4.4 *Structured output*

The text model can be made to answer in a fixed shape instead of prose. This is called *structured output*. It takes three parts.

structured output: the model must reply with data in a shape you define, not free text.

1. Describe the answer as a class. The same idea as `Index`: a container with named slots.

```
class SearchRequest(BaseModel):
    search_text: str                                  # the meaning part
    category: Literal["sofa", "mattress", ...] | None  # one of 8, or none
    max_price: float | None                           # a number, or none
```

2. Call the text model with the class as the schema. The class goes in as the `response_schema`, next to an instruction that says what to do:

```
reply = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="A red comfortable sofa under 5000 dkk",
    config=types.GenerateContentConfig(
        system_instruction="Split the customer's search into "
            "search_text and filters. Only set a filter if the "
            "customer clearly says it.",
        response_mime_type="application/json",
        response_schema=SearchRequest))
```

The `response_schema` forces the reply into that shape: valid JSON, a category from the eight allowed, a price that is a number. The model cannot answer "Sure! It looks like you want a sofa…".

3. Use the parts. The `search_text` goes to `embed_query`; `category` and `max_price` go into a `SearchFilters` for your filter function. Figure 3 shows the whole path.

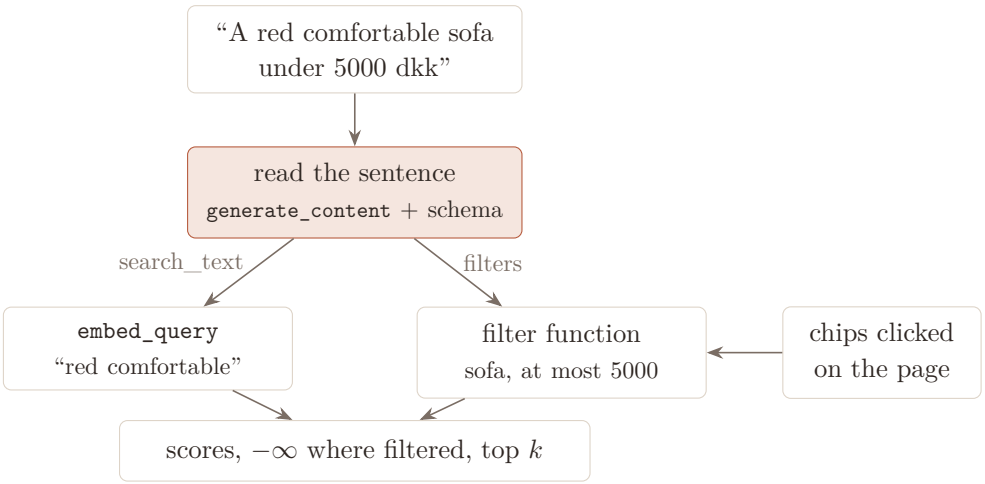


Figure 3 The highlighted step is new; the rest is Step 1 as you are writing it. Filters can arrive from the words, from the page, or both.

When the customer both clicks a filter and types one, the two are merged. A sensible rule is that the click wins, because it is the more deliberate of the two.

4.5 *It needs testing too*

The model reads well but not perfectly. Four searches, run once:

Search	Category	Max price	Comment
A red comfortable sofa under 5000 dkk	sofa	5000	right; text became “red comfortable”
billig madras under 2000 kr	mattress	2000	right; but “under 2000 kr” stayed in the text
something cosy for the living room floor	rug	none	a guess: rug was never said
pillow for neck pain	pillow	none	right

Table 5 Filters read from the words by `gemini-3-flash-preview`. “Billig madras” is Danish for “cheap mattress”.

The third row matters most. The instruction said “only if the customer clearly says it”, and the model guessed anyway. A filter is a hard cut, so a wrong filter hides every other product, however good. A wrong filter is worse than no filter. That is why this step gets its own test questions with known right answers, measured the same way as the search: the plan for Step 3.

Filters from the page are exact; filters from the words are read by a text model with structured output, before the search. Your filter function stays the same either way: it only ever sees a `SearchFilters`.

A All 20 graded questions

		as a query		as a document	
Question	Kind	p@5	top	p@5	top
a mattress that helps with back pain	meaning	1.0	0.737	1.0	0.801
madras der hjælper mod ondt i ryggen	language	1.0	0.788	0.8	0.832
Matratze gegen Rückenschmerzen	language	1.0	0.728	0.6	0.806
something soft to sleep on if I sleep on my side	meaning	1.0	0.712	0.4	0.752
mattress that does not get hot at night	meaning	1.0	0.698	1.0	0.778
pillow for neck pain	meaning	1.0	0.708	1.0	0.785
bedding for someone allergic to dust mites	meaning	1.0	0.681	1.0	0.763
varm dyne til vinteren	language	1.0	0.772	0.8	0.840
duvet for hot summer nights	meaning	1.0	0.725	0.8	0.799
a couch my guests can sleep on	meaning	1.0	0.716	1.0	0.778
Schlafsofa für Gäste	language	1.0	0.720	1.0	0.800
sofa that survives a dog and two kids	meaning	1.0	0.690	1.0	0.793
sofa where I can put my legs up	meaning	1.0	0.717	1.0	0.798
big table for family dinners that can get longer	meaning	1.0	0.728	1.0	0.814
spisebord i massiv eg	language	0.8	0.771	0.8	0.892
chair for working from home without back ache	meaning	1.0	0.702	1.0	0.812
outdoor furniture that can stay out in the rain	meaning	1.0	0.699	1.0	0.813
rug I can put in the washing machine	meaning	0.8	0.704	0.8	0.788
rug for my balcony	meaning	1.0	0.734	1.0	0.815
wool rug	meaning	1.0	0.704	1.0	0.809
Average		**0.98**	0.722	0.90	0.803

Table 6 Precision@5 and the highest score for each question, with the question embedded each way. Where the document type does worse, the number is marked. One run; the product vectors are the same in both columns.